المدرسة الوطنية للعلوم التطبيقية بتطوان
ⵜⵉⵍⵍⵉⵏ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵎⵓⵙⵙⴰⵙⵉⵏ ⵜⴰⵏⴰⵎⵓⵔⵜ
Ecole Nationale des Sciences Appliquées de Tétouan

National School of Applied Sciences of Tétouan
Department of Artificial Intelligence and Digitalization (IAD)
Data Science, Big Data, and AI

Gymnasium

Dynamic Programming Approaches for Solving the FrozenLake-v1 Environment

An Implementation Study Using Value Iteration and Policy Iteration

Prepared by:

Aymane Hayoun
Yassir Adila

Class: BDIA2

Supervised by:

Pr. A. Belcaid

Academic Year: 2025–2026

Contents

1 Introduction **2**

2 The MDP Framework **3**

3 Value Iteration Methodology **3**

4 Policy Iteration Methodology **4**

5 Implementation Analysis **5**

6 Results and Visualizations **6**

7 Comparative Study **6**
7.1 Value Iteration vs. Policy Iteration 6
7.2 Stochastic vs. Deterministic 7
7.3 4x4 vs. 8x8 Scaling 7

8 Comparative Summary **8**

9 Conclusion **8**

1 Introduction

Reinforcement Learning (RL) is a branch of machine learning focused on training agents to make sequential decisions in an environment to maximize cumulative rewards.
The FrozenLake-v1 environment from the Gymnasium library serves as an excellent testbed for classical RL methods due to its simple yet challenging grid-world setup.
In this environment, the agent navigates a frozen lake represented as a grid, starting from position 'S' (start), aiming to reach 'G' (goal) while avoiding 'H' (holes) that terminate the episode with zero reward, and traversing 'F' (frozen) tiles safely. The reward structure is sparse: +1 only upon reaching the goal, and 0 otherwise.

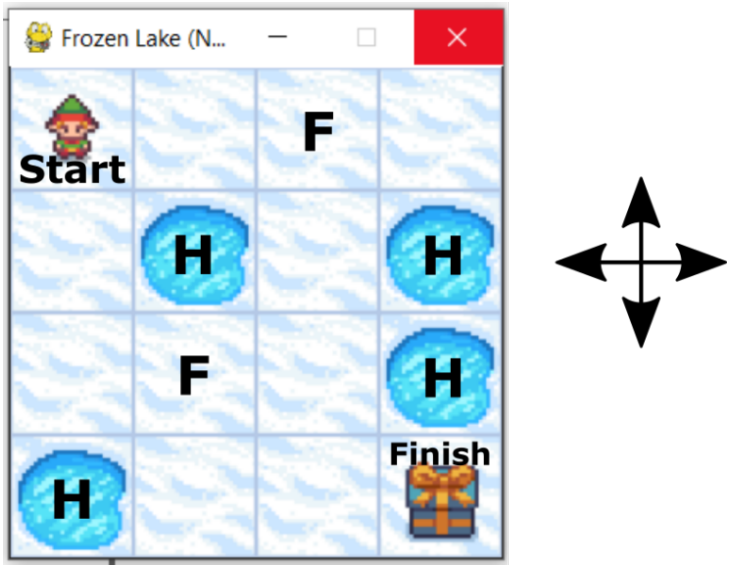


Figure 1: Visualization of the 4×4 FrozenLake-v1 environment

This project implements and analyzes two core Dynamic Programming (DP) algorithms **Value Iteration** (VI) and **Policy Iteration** (PI) to derive the optimal policy for the FrozenLake MDP. We use a discount factor $\gamma = 0.99$ to encourage long-term planning, a strict convergence threshold $\theta = 10^{-8}$ for precise value estimates, and evaluate policies over 1000 episodes for robust empirical performance assessment.
Experiments cover the standard 4x4 grid in both stochastic (is_slippery=True) and deterministic (is_slippery=False) modes, as well as the larger 8x8 stochastic grid to examine scalability.

The implementation, developed in a Google Colab notebook environment, leverages Gymnasium for the environment, NumPy for computations, and Matplotlib/Seaborn for visualizations. Key features include custom visualization tools that overlay state values on heatmaps with directional arrows representing the policy, providing intuitive insights into learned behaviors. This personalized study demonstrates the convergence properties of DP methods and highlights practical considerations in their application to tabular MDPs.

2 The MDP Framework

The FrozenLake environment is modeled as a finite Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$.

- $\mathcal{S}$: The state space consists of discrete grid positions, with $|\mathcal{S}| = 16$ for the 4x4 grid and 64 for the 8x8 grid. States are absorbing if they correspond to holes (failure) or the goal (success).
- $\mathcal{A}$: Four actions; 0: Left ($\leftarrow$), 1: Down ($\downarrow$), 2: Right ($\rightarrow$), 3: Up ($\uparrow$).
- $P(s'|s, a)$: Transition probabilities, accessed in the implementation via *env.unwrapped.P*, a nested dictionary providing lists of (probability, next_state, reward, done) tuples for each (s, a) pair.
- $R(s, a, s')$: Typically 0, except $+1$ when transitioning to the goal state.
- $\gamma = 0.99$: The discount factor used globally in the code.

The stochastic variant introduces non-determinism: for a chosen action a, the agent moves in the intended direction with probability 1/3, or perpendicularly left/right with 1/3 each. Boundary collisions result in staying in place, redistributing probabilities accordingly. Mathematically:

$$P(s' \mid s, a) = \frac{1}{3}$$

where each possible outcome (intended or perpendicular) is adjusted for boundaries.

This slipping mechanism, controlled by *is_ slippery*, significantly impacts policy robustness, as observed in our experiments.

3 Value Iteration Methodology

Value Iteration solves the Bellman optimality equation iteratively:

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma V^*(s')\right].$$

In our implementation, *value_ iteration* initializes $V(s) = 0$ for all states and performs synchronous updates. For each iteration, it computes action values by summing over transitions from *env.unwrapped.P[s][a]*, using the previous value estimate V_{old} to ensure stability. The update is:

$$V(s) \leftarrow \max_a \sum \text{prob} \cdot (\text{reward} + \gamma V_{\text{old}}(\text{next_state})).$$

Convergence is checked via the maximum delta across states, stopping when $\delta < 10^{-8}$. After convergence, the greedy policy is extracted using the final V.

This approach guarantees convergence to V^* due to the contraction property of the Bellman operator. In practice, for the 4x4 stochastic environment, convergence required 438 iterations, yielding $V(0) \approx 0.5420$.

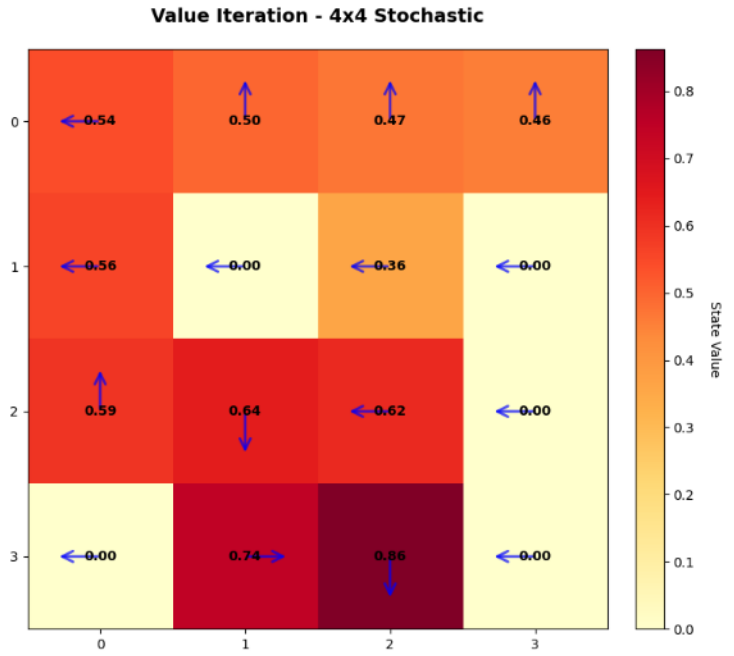


Figure 2: Optimal state-value heatmap and derived policy for the 4×4 stochastic Frozen-Lake environment after convergence via Value Iteration

4 Policy Iteration Methodology

Policy Iteration alternates between full policy evaluation and greedy improvement.

Policy Evaluation: For a fixed policy π, solve:

$$V^{\pi}(s) = \sum_{s'} P(s'|s, \pi(s)) \left[R(s, \pi(s), s') + \gamma V^{\pi}(s') \right].$$

Our *policy_ evaluation* function iteratively updates V using only the policy-dictated action per state, converging when changes fall below θ.

Policy Improvement: Compute:

$$\pi'(s) = \arg\max_{a} \sum_{s'} P(s'|s, a) \left[R(s, a, s') + \gamma V^{\pi}(s') \right],$$

again leveraging *env.unwrapped.P* for exact expectations.

The main *policy_ iteration* loop starts with a random policy (seeded for reproducibility), evaluates it fully, improves it greedily, and repeats until stability. This separation often results in fewer outer iterations compared to VI.

In our 4x4 stochastic run, Policy Iteration converged in 4 iterations to the same $V(0) \approx 0.5420$ as VI, with maximum value difference on the order of 10^{-9} and identical policies.

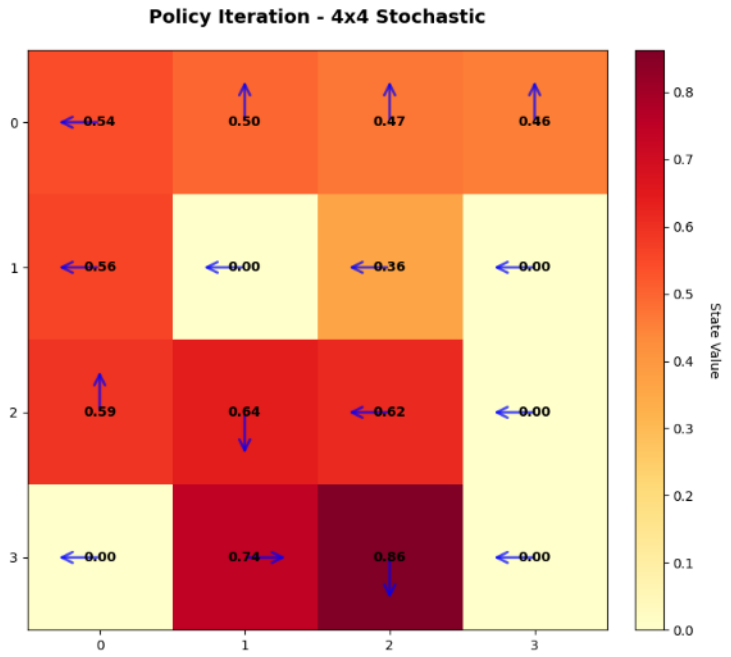


Figure 3: Optimal state-value heatmap and derived policy for the 4×4 stochastic Frozen-Lake environment after convergence via Policy Iteration.

5 Implementation Analysis

The project is organized into a modular notebook, where I defined global constants such as $\gamma = 0.99$ and $\theta = 10^{-8}$ at the top to ensure all experiments remain consistent. I used a `make_env` helper function to streamline the environment setup and ensure the results are reproducible across different runs.

To better interpret the agent's behavior, I developed a `plot_value_policy` function. This utility maps the V array onto the grid using a 'YlOrRd' heatmap and overlays the policy arrows using `FancyArrowPatch`. This visual approach is particularly useful for identifying specific states where the agent is likely to slip or succeed.

For the performance assessment, I implemented `evaluate_policy` to run 1,000 seeded episodes. By averaging the rewards and calculating the success rate, I was able to verify that the theoretical optimal policy holds up empirically. A significant advantage of this setup was the ability to access `env.unwrapped.P` directly. Since the state space is small, using the transition dynamics allowed for exact Dynamic Programming solutions without the need for the environmental sampling required by more complex MDPs.

6 Results and Visualizations

The visualizations clearly show how the state values flow backward from the goal, with values dropping significantly as they get closer to the holes. In the 4×4 stochastic environment, the optimal policy is visibly more cautious. Instead of taking the shortest path, the agent often chooses a detour to stay away from high-risk tiles next to holes. This makes sense because, in a world where the agent can slip, being "bold" or taking shortcuts usually leads to a zero reward.

The arrows in the policy plot consistently guide the agent toward the goal while giving hazards a wide berth, which demonstrates a learned aversion to risk. While the deterministic heatmaps show very smooth gradients, the stochastic ones are much more conservative. When we move to the 8×8 grid, this behavior becomes even more obvious; the agent takes much longer, circuitous routes to navigate the increased complexity and avoid falling in.

7 Comparative Study

7.1 Value Iteration vs. Policy Iteration

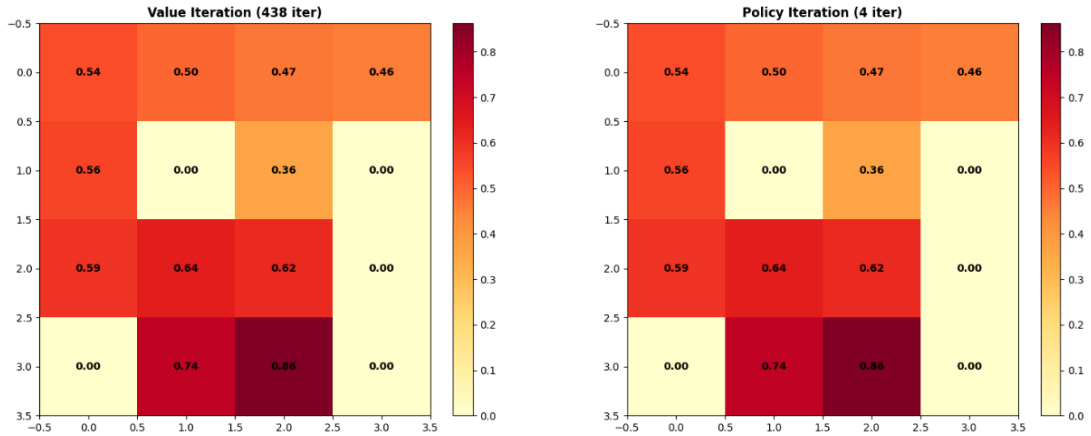


Figure 4: Comparison of converged state values for Value Iteration and Policy Iteration on the 4x4 stochastic grid

On the 4x4 stochastic environment, VI required 438 iterations, while PI converged in 4 iterations. Both achieved $V(0) \approx 0.542026$, with negligible difference ($0.0000000013 < 10^{-9}$) and identical policies. Empirical success rates were both 75.20%, confirming equivalence but highlighting PI's superior efficiency in iteration count stemming from complete evaluations per policy change.

7.2 Stochastic vs. Deterministic

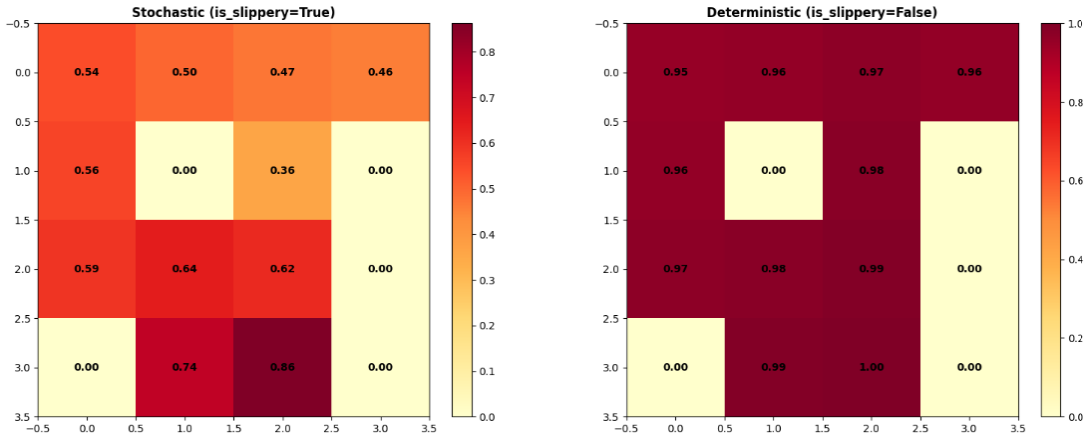


Figure 5: Comparison of state-value heatmaps between the stochastic (slippery) and deterministic versions of the 4x4 FrozenLake grid

Using VI on the 4x4 grid: stochastic yielded $V(0) \approx 0.542026$ as we already seen before. Deterministic required only 7 iterations and achieved $V(0) \approx 0.9510$, with 100% success rate versus 75.2% stochastic. The slipping mechanic substantially reduces state values and reliability, as unintended moves increase hole-fall probability.

7.3 4x4 vs. 8x8 Scaling

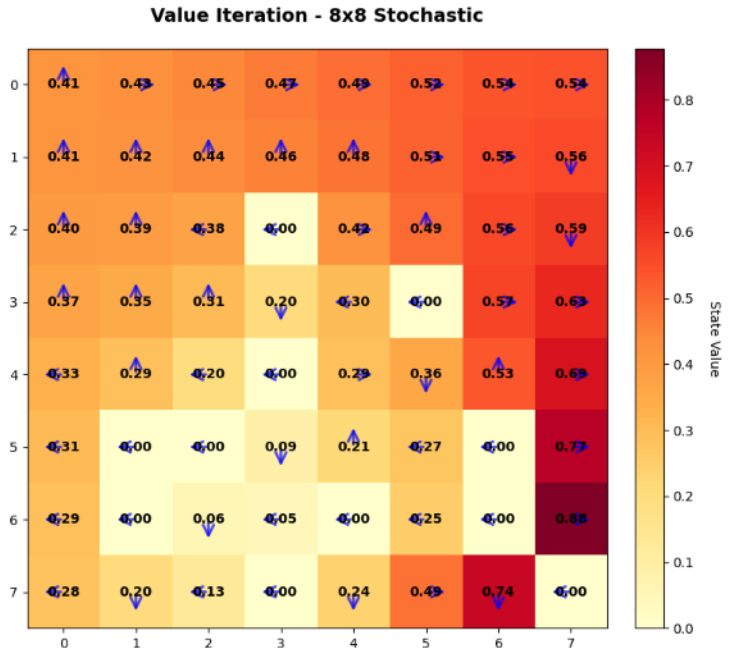


Figure 6: State-value heatmap and optimal policy for the 8×8 stochastic FrozenLake environment

VI on stochastic grids: 4x4 required 438 iterations ($V(0) \approx 0.542026$), 8x8 required 516 iterations ($V(0) \approx 0.414640$), with success rates 62.9 % vs. 75.2%. The larger state space

exacerbates the curse of dimensionality, ***more states prolong value propagation, while increased path complexity and slipping risks lower initial state value and performance.***

8 Comparative Summary

Table 1: Summary of FrozenLake Experimental Results

Environment	Algorithm	Stochastic	Iterations	$V(S)$	Success Rate	Avg Reward
4×4	Value Iteration	Yes	438	0.5420	75.20%	0.7520
4×4	Policy Iteration	Yes	4	0.5420	75.20%	0.7520
4×4	Value Iteration	No	7	0.9510	100.00%	1.0000
8×8	Value Iteration	Yes	516	0.4146	62.90%	0.6290

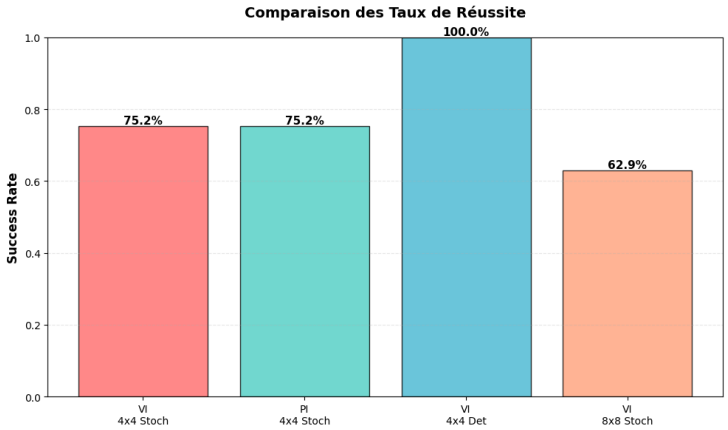


Figure 7: Comparative summary of agent success rates across different grid sizes, environment settings, and algorithms

9 Conclusion

Through this project, I have confirmed that both Value Iteration and Policy Iteration consistently find the optimal strategy for the FrozenLake environment. While both algorithms arrived at the exact same success rates and state values, Policy Iteration was significantly more efficient, reaching convergence in only 4 iterations compared to the 438 required by Value Iteration.

The experiments also highlighted how much the environment's physics dictate the agent's performance. The deterministic version was easily solved with a 100% success rate, but once we introduced the "slippery" stochastic mechanic, the success rate dropped to roughly 75%. Scaling up to the 8×8 grid further illustrated the challenges of larger state spaces; the agent had to take much longer paths, which increased the risk of slipping and brought the success rate down to 62.9%.

A few key takeaways from the implementation stand out: both algorithms produced identical optimal policies, and our heatmaps clearly visualized how the agent learned to navigate cautiously around hazards. While Dynamic Programming is extremely powerful

for these types of discrete problems where we have a full "map" of the environment, its reliance on transition knowledge is a clear limitation. For more complex or unknown environments, we would need to move toward model-free methods that learn purely from experience. Overall, this project provided a great hands-on foundation for understanding how planning and decision-making work in Reinforcement Learning.